

API Functionality: Register / Login

Full test run overview:

The screenshot displays a test runner interface for 'HustleHub+ Auth API'. The interface is divided into several sections:

- Left Sidebar:** Contains a list of collections and environments. The 'HustleHub+ Auth API' collection is expanded, showing various test cases like 'Health Check', 'Register - Valid', 'Register - Duplicate Email (Invalid)', etc.
- Top Bar:** Includes navigation icons, a search bar, and buttons for 'Invite', 'Upgrade', and 'No environment'.
- Overview Section:** Displays the test run results for 'HustleHub+ Auth API'. It shows a summary table with columns: Source, Environment, Iterations, Duration, All tests, Errors, and Avg. Resp. Time. The summary indicates 20 tests passed, 0 failed, and 0 skipped.
- Test Results Table:** A detailed table showing individual test results. Each row includes the test name, URL, status (PASS/FAIL), and duration. For example, 'POST Register - Valid' passed with a status of 201 and a duration of 611 ms.
- Bottom Bar:** Includes a status bar with 'Local mode', 'Terminal', 'Console', and 'Import Complete' buttons. It also shows the system clock and weather information.

Source	Environment	Iterations	Duration	All tests	Errors	Avg. Resp. Time
Runner	none	1	4s 151ms	20	0	-

Test Name	URL	Status	Duration	Response Size
POST Register - Valid	https://localhost:5443/api/auth/register	PASS	611 ms	1.487 KB
POST Register - Duplicate Email (Invalid)	https://localhost:5443/api/auth/register	FAIL	10 ms	1.101 KB
POST Register - Missing Fields (Invalid)	https://localhost:5443/api/auth/register	FAIL	7 ms	1.355 KB

1. Duplicate users

[illegible]

The screenshot displays the Postman interface with a collection named 'HustleHub+ Auth API'. The left sidebar lists various API endpoints, including 'Health Check', 'Register - Duplicate Email (Invalid)', 'Register - Missing Fields (Invalid)', 'Register - Malicious Input - Script Injection', 'Register - Weak Password (Invalid)', 'Register - Invalid Role', 'Login - Valid', 'Login - Missing Fields (Invalid)', 'Login - Wrong Password (Invalid)', 'Login - Nonexistent User (Invalid)', 'Get Current User - With Valid Token (Protect...', 'Get Current User - No Token (Invalid)', and 'Get Current User - Garbage Token (Invalid)'. The main panel shows the 'Register - Duplicate Email (Invalid)' test results, which passed with a 409 Conflict status. The response body is a JSON object indicating that an account with the provided email already exists.

Source	Environment	Iterations	Duration	All tests	Errors	Avg. Resp. Time
Runner	none	1	3s 947ms	20	0	-

POST Register - Duplicate Email (Invalid)
 https://localhost:5443/api/auth/register
 Status is 409 Conflict
 Generic error message returned

POST Register - Missing Fields (Invalid)
 https://localhost:5443/api/auth/register
 Status is 422 Unprocessable Entity
 Response does not expose internals

POST Register - Malicious Input - Script Injection
 https://localhost:5443/api/auth/register
 Status is 201 (accepted but sanitised)
 Script tags are escaped, not stored raw

POST Register - Weak Password (Invalid)
 https://localhost:5443/api/auth/register

POST HustleHub+ Auth API > Register - Duplicate Email (Invalid)
 Response: 409 - 7 ms - 1.1 KB
 Pretty

```

1 {
2   "success": false,
3   "message": "An account with this email already exists."
4 }
```

2.Invalid login (wrong password)

The screenshot shows the Postman interface with the 'HustleHub+ Auth API' collection. The 'Login - Wrong Password (Invalid)' test is selected, showing a 401 status code and a response body with email and password fields.

Test Results Summary:

Source	Environment	Iterations	Duration	All tests	Errors	Avg. Resp. Time
Runner	none	1	3s 947ms	20	0	-

Test Details:

- POST Login - Valid**
https://localhost:5443/api/auth/login
Status is 200
Response contains token
- POST Login - Missing Fields (Invalid)**
https://localhost:5443/api/auth/login
Status is 422 Unprocessable Entity
- POST Login - Wrong Password (Invalid)**
https://localhost:5443/api/auth/login
Status is 401
Generic error - does not reveal which field was wrong
- POST Login - Nonexistent User (Invalid)**
https://localhost:5443/api/auth/login
Status is 401

Response Body (JSON):

```
{  "email": "lindiwe.dube.1787749983582@gmail.com",  "password": "WrongPassword1"}
```

The screenshot shows the Postman interface with the 'HustleHub+ Auth API' collection. The 'Login - Wrong Password (Invalid)' test is selected, showing a 401 status code and a response body with success: false and message: 'Invalid email or password.'

Test Results Summary:

Source	Environment	Iterations	Duration	All tests	Errors	Avg. Resp. Time
Runner	none	1	3s 947ms	20	0	-

Test Details:

- POST Login - Valid**
https://localhost:5443/api/auth/login
Status is 200
Response contains token
- POST Login - Missing Fields (Invalid)**
https://localhost:5443/api/auth/login
Status is 422 Unprocessable Entity
- POST Login - Wrong Password (Invalid)**
https://localhost:5443/api/auth/login
Status is 401
Generic error - does not reveal which field was wrong
- POST Login - Nonexistent User (Invalid)**
https://localhost:5443/api/auth/login
Status is 401

Response Body (JSON):

```
{  "success": false,  "message": "Invalid email or password."}
```

3. Missing fields on registration

The screenshot shows the Postman interface with the 'HustleHub+ Auth API' collection selected. The 'Run results' tab is active, displaying a table of test results. The 'Register - Missing Fields (Invalid)' test is highlighted, showing a 422 status code and a JSON response indicating a missing email field.

Source	Environment	Iterations	Duration	All tests	Errors	Avg. Resp. Time
Runner	none	1	3s 947ms	20	0	-

POST Register - Missing Fields (Invalid)
https://localhost:5443/api/auth/register
Status is 422 Unprocessable Entity
Response does not expose internals

Response
1 {
2 |
3 | "email": "missingfields.test@gmail.com"
4 }

The screenshot shows the Postman interface with the 'HustleHub+ Auth API' collection selected. The 'Run results' tab is active, displaying a table of test results. The 'Register - Missing Fields (Invalid)' test is highlighted, showing a 422 status code and a JSON response indicating a missing name field.

Source	Environment	Iterations	Duration	All tests	Errors	Avg. Resp. Time
Runner	none	1	3s 947ms	20	0	-

POST Register - Missing Fields (Invalid)
https://localhost:5443/api/auth/register
Status is 422 Unprocessable Entity
Response does not expose internals

Response
1 {
2 |
3 | "success": false,
4 | "message": "Name is required. Name must be between 2 and 100
characters. Password is required. Password must be at least 8
characters long. Password must contain at least one number.
Password must contain at least one uppercase letter. Role is
required. Role must be one of: freelancer, client."
5 }

Missing fields on login

Postman interface showing API test results for HustleHub+ Auth API. The interface includes a sidebar with collections, a main panel with test results, and a bottom status bar.

Test Results Summary:

Source	Environment	Iterations	Duration	All tests	Errors	Avg. Resp. Time
Runner	none	1	3s 947ms	20	0	-

Test Details:

- POST Login - Valid**
https://localhost:5443/api/auth/login
Status is 200
Response contains token
- POST Login - Missing Fields (Invalid)**
https://localhost:5443/api/auth/login
Status is 422 Unprocessable Entity
- POST Login - Wrong Password (Invalid)**
https://localhost:5443/api/auth/login
Status is 401
Generic error - does not reveal which field was wrong
- POST Login - Nonexistent User (Invalid)**
https://localhost:5443/api/auth/login
Status is 401

Response Body (Pretty):

```
1 {}
```

Postman interface showing API test results for HustleHub+ Auth API. The interface includes a sidebar with collections, a main panel with test results, and a bottom status bar.

Test Results Summary:

Source	Environment	Iterations	Duration	All tests	Errors	Avg. Resp. Time
Runner	none	1	3s 947ms	20	0	-

Test Details:

- POST Login - Valid**
https://localhost:5443/api/auth/login
Status is 200
Response contains token
- POST Login - Missing Fields (Invalid)**
https://localhost:5443/api/auth/login
Status is 422 Unprocessable Entity
- POST Login - Wrong Password (Invalid)**
https://localhost:5443/api/auth/login
Status is 401
Generic error - does not reveal which field was wrong
- POST Login - Nonexistent User (Invalid)**
https://localhost:5443/api/auth/login
Status is 401

Response Body (Pretty):

```
1 {  
2   "success": false,  
3   "message": "Email is required. A valid email address is required.  
4             Password is required."  
}
```

Clean routing and controller separation

The controller file was inspected to confirm that all registration and login logic (checking for duplicates, hashing passwords, issuing tokens) lives there instead, keeping each file focused on a single responsibility.

```
MINGW64~/c/Users/Admin/hustlehub-plus/backend
Admin@DESKTOP-42UKV3K MINGW64 ~/hustlehub-plus/backend (master)
$ cat src/controllers/authController.js
const userStore = require('../models/userStore');
const { hashPassword, comparePassword } = require('../utils/password');
const { signToken } = require('../utils/token');
const AppError = require('../utils/AppError');

async function register(req, res, next) {
  try {
    const { name, email, password, role } = req.body;

    const existingUser = userStore.findByEmail(email);
    if (existingUser) {
      return next(new AppError('An account with this email already exists.', 409));
    }

    const passwordHash = await hashPassword(password);
    const newUser = userStore.create({ name, email, passwordHash, role });

    const token = signToken(newUser);

    res.status(201).json({
      success: true,
      message: 'User registered successfully.',
      data: {
        user: userStore.toPublicUser(newUser),
        token,
      },
    });
  } catch (err) {
    next(err);
  }
}

async function login(req, res, next) {
  try {
    const { email, password } = req.body;

    const user = userStore.findByEmail(email);

    const genericAuthError = () => new AppError('Invalid email or password.', 401);

    if (!user) {
      return next(genericAuthError());
    }

    const passwordMatches = await comparePassword(password, user.passwordHash);
    if (!passwordMatches) {
      return next(genericAuthError());
    }
  }
}
```

22°C Sunny 12:22 PM 8/27/2026

```
MINGW64/c/Users/Admin/hustlehub-plus/backend
const { email, password } = req.body;
const user = userStore.findByEmail(email);

const genericAuthError = () => new AppError('Invalid email or password.', 401);

if (!user) {
  return next(genericAuthError());
}

const passwordMatches = await comparePassword(password, user.passwordHash);
if (!passwordMatches) {
  return next(genericAuthError());
}

const token = signToken(user);

res.status(200).json({
  success: true,
  message: 'Login successFull.',
  data: {
    user: userStore.toPublicUser(user),
    token,
  },
});
} catch (err) {
  next(err);
}
}

async function getCurrentUser(req, res, next) {
  try {
    const user = userStore.findById(req.user.id);
    if (!user) {
      return next(new AppError('User not found.', 404));
    }
    res.status(200).json({
      success: true,
      data: { user: userStore.toPublicUser(user) },
    });
  } catch (err) {
    next(err);
  }
}

module.exports = { register, login, getCurrentUser };

Admin@DESKTOP-42UKV3K MINGW64 ~/hustlehub-plus/backend (master)
$
```


Password Security & JWT

Passwords hashed:

Every user record contains a bcrypt password hash rather than the original password, confirming that passwords are never stored in a readable format.

```
Admin@DESKTOP-42UKV3K MINGW64 ~/hustlehub-plus/postman (master)
$ node -e "console.log(JSON.parse(require('fs').readFileSync('HustleHub-Auth.postman_collection.json')).item.length)"
14

Admin@DESKTOP-42UKV3K MINGW64 ~/hustlehub-plus/postman (master)
$ cat ~/hustlehub-plus/backend/src/data/users.json
[
  {
    "id": "b7843200-0902-477c-98a7-26432ade98c1",
    "name": "Lindiwe Dube",
    "email": "lindiwedube@gmail.com",
    "passwordHash": "$2b$12$Q10v31xrxz968XWAmgV/nutcnZ5LJIdKG0qIiIm9hu0xcamfCkCMS",
    "role": "freelancer",
    "createdAt": "2026-08-26T13:08:38.429Z"
  },
  {
    "id": "9316f532-ffd4-4e83-a0cd-828d13e75091",
    "name": "&lt;script&gt;alert(&#x27;xss&#x27;)&lt;&#x2F;script&gt;;",
    "email": "maliciousinputtest@gmail.com",
    "passwordHash": "$2b$12$J8QNVUTwNLn3lQyR9RJsc.ngW0SoGbwW7g2PdVp09SaYD6MqvzzPG",
    "role": "client",
    "createdAt": "2026-08-26T13:08:39.542Z"
  },
  {
    "id": "d234a6a4-533e-4b2d-b19b-22a16a7df097",
    "name": "Lindiwe Dube",
    "email": "lindiwedubel787749983582@gmail.com",
    "passwordHash": "$2b$12$7V9ifohq5impNDRuMlKQeUtCWJMI7AE3ZPSPCKF20AsQt47oRn1Q.",
    "role": "freelancer",
    "createdAt": "2026-08-26T13:13:04.053Z"
  },
  {
    "id": "e0cbeff3-5fe7-4b3a-8c91-e124fca1442e",
    "name": "&lt;script&gt;alert(&#x27;xss&#x27;)&lt;&#x2F;script&gt;;",
    "email": "maliciousinputl787749984401@gmail.com",
    "passwordHash": "$2b$12$3p3tshW0gFFLCGUHjhXdm.9G0.8jDBS3GyhbHCwkUgz3381BUGbsy",
    "role": "client",
    "createdAt": "2026-08-26T13:13:04.911Z"
  }
]
```



Hashed passwords verified correctly (both directions)

A login attempt with the correct password succeeded and returned a token, while a login attempt with an incorrect password for the same account was rejected. This confirms the password comparison genuinely checks the submitted password against the stored hash, rather than always allowing or always rejecting access.

Nobesuthu Gongo's Workspace

Collections

- HustleHub+ Auth API
 - GET Health Check
 - POST Register - Valid
 - POST Register - Duplicate Email (Invalid)
 - POST Register - Missing Fields (Invalid)
 - POST Register - Malicious Input - Script Injection
 - POST Register - Weak Password (Invalid)
 - POST Register - Invalid Role
 - POST Login - Valid
 - POST Login - Missing Fields (Invalid)
 - POST Login - Wrong Password (Invalid)
 - POST Login - Nonexistent User (Invalid)
 - GET Get Current User - With Valid Token (Protect...)
 - GET Get Current User - No Token (Invalid)
 - GET Get Current User - Garbage Token (Invalid)

Environments

Documents

Specs

Mocks

Flows

Overview

HustleHub+ Auth API

Ran today at 03:13:02 PM

Source	Environment	Iterations	Duration	All tests	Errors	Avg. Resp. Time
Runner	none	1	3s 947ms	20	0	-

All 20 Passed 20 Failed 0 Skipped 0 Errors 0 Console log

POST HustleHub+ Auth API > Login - Valid

Response Headers Request

200 - 468 ms - 1.482 KB

1 {
2 "success": true,
3 "message": "Login successful.",
4 "data": {
5 "user": {
6 "id": "d234a6a4-533e-4b2d-b19b-22a16a7df097",
7 "name": "Lindiwe Dube",
8 "email": "lindiwedube1787749983582@gmail.com",
9 "role": "freelancer",
10 "createdAt": "2026-08-26T13:13:04.053Z"
11 },
12 "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.
eyJpc2UiOiMzMmVlbG9yYyIiwiaWF0IjoxNzg3NDQ5OTg1LCJleHAiOj
E3ODc3NTMTODV9.3qsrt2Y6tulXBietLfzrLM7nuTR84GTquUAxCtp9gDQ"
13 }
14 }

Local mode

Terminal

Console

Import Complete

19°C Mostly cloudy

Search

3:37 PM 8/26/2026

The screenshot displays the Postman interface with the 'HustleHub+ Auth API' collection. The 'Run results' tab shows a table of test outcomes:

Source	Environment	Iterations	Duration	All tests	Errors	Avg. Resp. Time
Runner	none	1	3s 947ms	20	0	-

Below the table, individual test results are listed. The 'Login - Wrong Password (Invalid)' test is selected, showing a 401 status and the following JSON response:

```
{
  "success": false,
  "message": "Invalid email or password."
}
```

JWT contains appropriate payload

A token issued during login was decoded to inspect its contents. The payload contains only a user ID, a role, and standard timestamp fields no email, name, or password information is included, since a JWT can be read by anyone who holds it.

```
Admin@DESKTOP-42UKV3K MINGW64 ~/hustlehub-plus/postman (master)
$ curl -sk -X POST https://localhost:5443/api/auth/login \
-H "Content-Type: application/json" \
-d '{"email":"lindiwe.dube@gmail.com","password":"PasswOrd123"}'
{"success":true,"message":"Login successful.", "data":{"user":{"id":"75ba1888-9b25-43cb-8734-bbfeaea24b72","name":"Lindiwe Dube","email":"lindiwedube@gmail.com","role":"freelancer","createdAt":"2026-08-23T12:15:03.552Z"},"token":"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6IjclYmExODg4LTl1MjUtNDNjYi04Nm0wLWJiZWZhZWEyNGI3MiIsInJvbmU0IjpmcmV1bGFuY2V5IiwiaWF0IjoxNzg3NDg5MDcyLCJ1eHAiOiJlE3ODc0OTI2NzJ9.qkwBh5S5Jf7CQt3iRaItIfb3vkQ4RNxw9fs-4AnvZg"}}
Admin@DESKTOP-42UKV3K MINGW64 ~/hustlehub-plus/postman (master)
$ cd ~/hustlehub-plus/backend
node -e "console.log(require('jsonwebtoken').decode('eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6IjclYmExODg4LTl1MjUtNDNjYi04Nm0wLWJiZWZhZWEyNGI3MiIsInJvbmU0IjpmcmV1bGFuY2V5IiwiaWF0IjoxNzg3NDg5MDcyLCJ1eHAiOiJlE3ODc0OTI2NzJ9.qkwBh5S5Jf7CQt3iRaItIfb3vkQ4RNxw9fs-4AnvZg'))"
{
  id: '75ba1888-9b25-43cb-8734-bbfeaea24b72',
  role: 'freelancer',
  iat: 1787489072,
  exp: 1787492672
}
Admin@DESKTOP-42UKV3K MINGW64 ~/hustlehub-plus/backend (master)
$
```

Secure secret usage

The server was configured to validate the JWT secret during startup. To verify this functionality, the placeholder secret from the example .env file was temporarily restored and the server was restarted. The server correctly refused to start and displayed an error indicating that an insecure placeholder secret was being used. The genuine secret was then restored, after which the server started successfully. This confirmed that the validation mechanism correctly blocks insecure placeholder values while allowing the server to operate normally when a secure secret is provided.

```
npm start
GET /api/users/me 401 0.767 ms - 54

Admin@DESKTOP-42UKV3K MINGW64 ~/hustlehub-plus/backend (master)
$ cd ~/hustlehub-plus/backend

Admin@DESKTOP-42UKV3K MINGW64 ~/hustlehub-plus/backend (master)
$ cp .env .env.backup

Admin@DESKTOP-42UKV3K MINGW64 ~/hustlehub-plus/backend (master)
$ sed -i 's/JWT_SECRET=,/JWT_SECRET=replace_this_with_a_long_random_string_before_running/' .env

Admin@DESKTOP-42UKV3K MINGW64 ~/hustlehub-plus/backend (master)
$ npm start

> backend@1.0.0 start
> node src/server.js

o injected env (9) from .env // tip: * multiple files { path: ['.env.local', '.env'] }
C:\Users\Admin\hustlehub-plus\backend\src\config\env.js:15
  throw new Error(
    ^
Error: JWT_SECRET is still set to the placeholder value from .env.example. Set a strong, unique secret before starting the server.
    at assertRequiredEnv (C:\Users\Admin\hustlehub-plus\backend\src\config\env.js:15:11)
    at Object.<anonymous> (C:\Users\Admin\hustlehub-plus\backend\src\config\env.js:22:1)
    at Module._compile (node:internal/modules/cjs/loader:1706:14)
    at Object..js (node:internal/modules/cjs/loader:1839:10)
    at Module.load (node:internal/modules/cjs/loader:1441:32)
    at Function._load (node:internal/modules/cjs/loader:1263:12)
    at TracingChannel.traceSync (node:diagnostics_channel:322:14)
    at wrapModuleload (node:internal/modules/cjs/loader:237:24)
    at Module.require (node:internal/modules/cjs/loader:1463:12)
    at require (node:internal/modules/helpers:147:16)

Node.js v22.19.0

Admin@DESKTOP-42UKV3K MINGW64 ~/hustlehub-plus/backend (master)
$ mv .env.backup .env

Admin@DESKTOP-42UKV3K MINGW64 ~/hustlehub-plus/backend (master)
$ npm start

> backend@1.0.0 start
> node src/server.js

o injected env (9) from .env // tip: * enable debugging { debug: true }
HustleHub+ API listening securely on https://localhost:5443
```

Token structure, consistent and ready for route protection

A protected endpoint was called with a valid token attached in the request's Authorization header, using the standard "Bearer" format. The request succeeded and returned the logged-in user's profile, confirming the token is correctly structured, transmitted, and enforced before access is granted.

HTTPS Implementation

Server running securely

When the server starts, the console displays HustleHub+ API listening securely on `https://localhost:5443`. This confirms that the application was configured to use the certificate and private key and to run directly over HTTPS.

```
$ cd ~/hustlehub-plus/backend
Admin@DESKTOP-42UKV3K MINGW64 ~/hustlehub-plus/backend (master)
$ npm start

> backend@1.0.0 start
> node src/server.js

◇ injected env (9) from .env // tip: ✦ secrets for agents [www.dotenvx.com]
HustleHub+ API listening securely on https://localhost:5443
|
```



Real certificate in use

A TLS certificate (`cert.pem`) and private key (`key.pem`) were created and safely stored in the backend's `certs/` folder. When a normal HTTP request was sent to the HTTPS port, the server rejected it. This shows that the server only accepts secure HTTPS connections and does not allow unencrypted connections.

```
Admin@DESKTOP-42UKV3K MINGW64 ~/hustlehub-plus/postman (master)
$ ls -la ~/hustlehub-plus/backend/certs
total 12
drwxr-xr-x 1 Admin 197121  0 Aug 20 21:14 ./
drwxr-xr-x 1 Admin 197121  0 Aug 26 15:44 ../
-rw-r--r-- 1 Admin 197121 1024 Aug 20 21:14 cert.pem
-rw-r--r-- 1 Admin 197121 1704 Aug 20 21:14 key.pem

Admin@DESKTOP-42UKV3K MINGW64 ~/hustlehub-plus/postman (master)
$ curl http://localhost:5443/api/health
curl: (52) Empty reply from server

Admin@DESKTOP-42UKV3K MINGW64 ~/hustlehub-plus/postman (master)
$ |
```



Successful handshake & security headers

The `-k` flag was used with `curl` to bypass certificate verification, because the certificate is self-signed rather than issued by a recognised authority. This is expected for local development and does not indicate the HTTPS setup is broken in production, a certificate from a trusted CA would be used instead. The verbose `curl` output confirms the secure handshake completed successfully before any data was exchanged, and the server responded with 200 OK along with a set of security headers.

```
MINGW64/c/Users/Admin/hustlehub-plus/postman
Admin@DESKTOP-42UKV3K MINGW64 ~/hustlehub-plus/postman (master)
$ curl -v -k https://localhost:5443/api/health
* Host localhost:5443 was resolved.
* IPv6: ::1
* IPv4: 127.0.0.1
* Trying ::1:5443...
* schannel: disabled automatic use of client certificate
* ALPN: curl offers http/1.1
* ALPN: server accepted http/1.1
* Established connection to localhost (::1 port 5443) from ::1 port 59950
* using HTTP/1.x
> GET /api/health HTTP/1.1
> Host: localhost:5443
> User-Agent: curl/8.19.0
> Accept: */*
>
* Request completely sent off
* schannel: remote party requests renegotiation
* schannel: renegotiating SSL/TLS connection
* schannel: SSL/TLS connection renegotiated
* schannel: remote party requests renegotiation
* schannel: renegotiating SSL/TLS connection
* schannel: SSL/TLS connection renegotiated
< HTTP/1.1 200 OK
< Content-Security-Policy: default-src 'self';base-uri 'self';font-src 'self' https: data;;form-action 'self';frame-ancestors 'self';img-src 'self' data;;object-src 'none';script-src 'self';script-src-attr 'none';style-src 'self' https: 'unsafe-inline';upgrade-insecure-requests
< Cross-Origin-Opener-Policy: same-origin
< Cross-Origin-Resource-Policy: same-origin
< Origin-Agent-Cluster: ?1
< Referrer-Policy: no-referrer
< Strict-Transport-Security: max-age=31536000; includeSubDomains
< X-Content-Type-Options: nosniff
< X-DNS-Prefetch-Control: off
< X-Download-Options: noopen
< X-Frame-Options: SAMEORIGIN
< X-Permitted-Cross-Domain-Policies: none
< X-XSS-Protection: 0
< Vary: Origin
< Access-Control-Allow-Credentials: true
< Content-Type: application/json; charset=utf-8
< Content-Length: 55
< ETag: W/"37-XswjRZi9Q5GrYMSyNqSPzeibgAU"
< Date: Wed, 28 Aug 2024 14:44:11 GMT
< Connection: keep-alive
< Keep-Alive: timeout=5
<
{"success":true,"message":"HustleHub+ API is running."}
* Connection #0 to host localhost:5443 left intact
```

18°C
Mostly cloudy

Search



4:44 PM
8/26/2024

Input Validation & Error Handling:

Invalid input

A registration request was sent with an email address that was present but not in a valid format

This screenshot shows the Postman interface for a REST client. The left sidebar lists a collection of API tests under 'HustleHub+ Auth API'. The selected test is 'POST Register - Invalid Role'. The main panel shows the request details for a POST request to 'HustleHub+ Auth API > Register - Invalid Role' with the URL 'POST {{baseUri}}/auth/register'. The request body is a JSON object:

```
{  "name": "Test",  "email": "not-an-email-address",  "password": "Passw@rd123",  "role": "client"}
```

. The response tab shows a '422 Unprocessable Entity (WebDAV) (RFC 4918)' status with a message:

```
{  "success": false,  "message": "A valid email address is required."}
```

. The bottom status bar indicates the test 'Import Complete'.

This screenshot shows the same Postman interface as above, but with the 'Test Results' tab selected. The test results show a 'PASSED' status with the message 'Status is 422 Unprocessable Entity'. The response body is the same as in the previous screenshot:

```
{  "success": false,  "message": "A valid email address is required."}
```

. The bottom status bar indicates the test 'Import Complete'.

The screenshot displays a REST client interface for testing an API. The interface is split into three main sections: a left sidebar with a collection of tests, a middle panel showing the results of a specific test run, and a right panel showing the raw response data.

Left Sidebar (Collections):

- HustleHub+ Auth API**
 - GET Health Check
 - POST Register - Valid
 - POST Register - Duplicate Email (Invalid)
 - POST Register - Missing Fields (Invalid)
 - POST Register - Malicious Input - Script Injection
 - POST Register - Weak Password (Invalid)
 - POST Register - Invalid Role
 - POST Login - Valid
 - POST Login - Missing Fields (Invalid)
 - POST Login - Wrong Password (Invalid)
 - POST Login - Nonexistent User (Invalid)
 - GET Get Current User - With Valid Token (Protect...)
 - GET Get Current User - No Token (Invalid)
 - GET Get Current User - Garbage Token (Invalid)
- ENVIRONMENTS**
- DOCUMENTS**
- SPECS**
- MOCKS**
- DATASETS**
- FLOWS**

Middle Panel (HustleHub+ Auth API - Run results):

Ran today at 01:21:37 PM

Source	Environment	Iterations	Duration	All tests	Errors	Avg. Resp. Time
Runner	none	1	3s 565ms	20	0	-

All 20 Passed 20 Failed 0 Skipped 0 Errors 0 Console log

POST Register - Missing Fields (Invalid)
https://localhost:5443/api/auth/register
PASS Status is 422 Unprocessable Entity
PASS Response does not expose internals

POST Register - Malicious Input - Script Injection
https://localhost:5443/api/auth/register
201 2
PASS Status is 201 (accepted but sanitised)
PASS Script tags are escaped, not stored raw

POST Register - Weak Password (Invalid)
https://localhost:5443/api/auth/register
422 1
PASS Status is 422 Unprocessable Entity

POST Register - Invalid Role
https://localhost:5443/api/auth/register
422 1
PASS Status is 422 Unprocessable Entity

Right Panel (HustleHub+ Auth API > Register - Malicious Input - Script Injection):

Response Headers Request

201 - 424 ms - 1.537 KB

Pretty

```
1 {
2   "name": "<script>alert('xss')</script>",
3   "email": "malicious.input.178800249879@gmail.com",
4   "password": "Password123",
5   "role": "client"
6 }
```

Handles Missing:

Postman interface showing the 'HustleHub+ Auth API' collection. The 'Run results' tab is active, displaying a table of test results. The 'POST Register - Missing Fields (Invalid)' test is selected, showing a response with a 422 status code and a message indicating missing fields.

Source	Environment	Iterations	Duration	All tests	Errors	Avg. Resp. Time
Runner	none	1	3s 947ms	20	0	-

POST Register - Missing Fields (Invalid)
https://localhost:5443/api/auth/register
422 2

Response Headers Request

```
1 {  
2   "email": "missingfields.test@gmail.com"  
3 }
```

Postman interface showing the 'HustleHub+ Auth API' collection. The 'Run results' tab is active, displaying a table of test results. The 'POST Register - Missing Fields (Invalid)' test is selected, showing a response with a 422 status code and a message indicating missing fields.

Source	Environment	Iterations	Duration	All tests	Errors	Avg. Resp. Time
Runner	none	1	3s 947ms	20	0	-

POST Register - Missing Fields (Invalid)
https://localhost:5443/api/auth/register
422 2

Response Headers Request

```
1 {  
2   "success": false,  
3   "message": "Name is required. Name must be between 2 and 100  
4             characters. Password is required. Password must be at least 8  
              characters long. Password must contain at least one number.  
              Password must contain at least one uppercase letter. Role is  
              required. Role must be one of: freelancer, client."  
5 }
```

Error handling:

Two scenarios were tested to confirm errors are handled safely rather than exposing internal details. A request to a URL that does not exist returned the same consistent error format used everywhere else in the API. A request with an intentionally oversized field, around 20,000 characters, was rejected with a specific, clear message instead of crashing the server or returning a vague failure.

```
Admin@DESKTOP-42UKV3K MINGW64 ~/hustlehub-plus/backend (master)
$ curl -sk https://localhost:5443/api/this-route-does-not-exist
{"success":false,"message":"The requested resource was not found."}
Admin@DESKTOP-42UKV3K MINGW64 ~/hustlehub-plus/backend (master)
$ |
```

```
Admin@DESKTOP-42UKV3K MINGW64 ~/hustlehub-plus/backend (master)
$ BIGNAME=$(printf 'A%.0s' {1..20000})
curl -sk -X POST https://localhost:5443/api/auth/register \
  -H "Content-Type: application/json" \
  -d "{\"name\":\"$BIGNAME\",\"email\":\"big@gmail.com\",\"password\":\"Passw0rd123\",\"role\":\"client\"}"
{"success":false,"message":"Request body is too large."}
Admin@DESKTOP-42UKV3K MINGW64 ~/hustlehub-plus/backend (master)
```

19°C
Mostly cloudy



Search



Code structure:

The backend folder structure was reviewed to confirm routes, controllers, middleware, models, and utility functions are each kept in separate, clearly named files. Shared logic, such as the authentication check and the custom error type, was found to be written once and reused across multiple files rather than duplicated. File sizes were also checked, confirming each file remains small and focused on a single responsibility, which keeps the codebase easy to navigate and modify.

```
Admin@DESKTOP-42UKV3K MINGW64 ~/hustlehub-plus/backend (master)
$ ls -R src
src:
app.js  config/  controllers/  data/  middleware/  models/  routes/  server.js  utils/

src/config:
env.js

src/controllers:
authController.js

src/data:
users.json

src/middleware:
authenticate.js  errorHandler.js  rateLimiter.js  validators.js

src/models:
userStore.js

src/routes:
authRoutes.js  userRoutes.js

src/utils:
AppError.js  password.js  token.js

Admin@DESKTOP-42UKV3K MINGW64 ~/hustlehub-plus/backend (master)
$ cat src/routes/authRoutes.js
const express = require('express');
const { register, login } = require('../controllers/authController');
const { registerRules, loginRules, handleValidationErrors } = require('../middleware/validators');
const { authLimiter } = require('../middleware/rateLimiter');

const router = express.Router();

router.post('/register', authLimiter, registerRules, handleValidationErrors, register);
router.post('/login', authLimiter, loginRules, handleValidationErrors, login);

module.exports = router;

Admin@DESKTOP-42UKV3K MINGW64 ~/hustlehub-plus/backend (master)
$ |
```

```
Admin@DESKTOP-42UKV3K MINGW64 ~/hustlehub-plus/backend (master)
$ grep -rn "authenticate" src/routes/
src/routes/userRoutes.js:3:const authenticate = require('../middleware/authenticate');
src/routes/userRoutes.js:7:router.get('/me', authenticate, getCurrentUser);

Admin@DESKTOP-42UKV3K MINGW64 ~/hustlehub-plus/backend (master)
$ grep -rn "AppError" src/
src/controllers/authController.js:4:const AppError = require('../utils/AppError');
src/controllers/authController.js:12:    return next(new AppError('An account with this email already exists.', 409));
src/controllers/authController.js:39:    const genericAuthError = () => new AppError('Invalid email or password.', 401);
src/controllers/authController.js:69:    return next(new AppError('User not found.', 404));
src/middleware/authenticate.js:2:const AppError = require('../utils/AppError');
src/middleware/authenticate.js:10:    return next(new AppError('Authentication required.', 401));
src/middleware/authenticate.js:18:    return next(new AppError('Authentication required.', 401));
src/middleware/authenticate.js:24:    return next(new AppError('Invalid or expired token.', 401));
src/middleware/validators.js:2:const AppError = require('../utils/AppError');
src/middleware/validators.js:53:    return next(new AppError(messages.join(' '), 422));
src/utils/AppError.js:1:class AppError extends Error {
src/utils/AppError.js:10:module.exports = AppError;

Admin@DESKTOP-42UKV3K MINGW64 ~/hustlehub-plus/backend (master)
$ |
```

```
Admin@DESKTOP-42UKV3K MINGW64 ~/hustlehub-plus/backend (master)
```

```
$ wc -l src/**/*.js src/*.js 2>/dev/null | sort -n
```

```
 9 src/routes/userRoutes.js
10 src/utls/AppError.js
11 src/routes/authRoutes.js
13 src/utls/password.js
14 src/middleware/rateLimiter.js
16 src/utls/token.js
28 src/middleware/authenticate.js
33 src/middleware/errorHandler.js
35 src/config/env.js
36 src/app.js
38 src/server.js
58 src/middleware/validators.js
68 src/models/userStore.js
80 src/controllers/authController.js
449 total
```

```
Admin@DESKTOP-42UKV3K MINGW64 ~/hustlehub-plus/backend (master)
```

```
$ |
```